Programming Language I (CSE110) [Last Modified : 25 May, 2025]

STRING

1. Write a program to take a string as input and print it.
2. Find the length of a given string without using length().
3. Convert a given string to uppercase and lowercase.
4. Concatenate two strings without using the + operator.
5. Check if a given string is empty or not.
6. Compare two strings using == and.equals() and explain the difference.
7. Check if a string contains a given substring.
8. Find the first and last occurrence of a character in a string.
9. Find the number of occurrences of a character in a string.
10. Reverse a string without using built-in methods.
11. Remove all vowels from a given string.
12. Replace all spaces in a string with -.
13. Remove duplicate characters from a string.
14. Count the number of words in a string.
15. Convert a string to a character array and print each character.
16. Convert a string representation of a number to an integer ("123" → 123).
17. Convert a character array back into a string.
18. Split a string based on spaces or a given delimiter.
19. Convert the first letter of each word in a string to uppercase.
20. Check if a string is a palindrome.
21. Find the longest word in a given sentence.
22. Extract digits from a given string ("a1b2c3" → 123).
23. Find and replace a substring in a string.
24. Sort the characters of a string in alphabetical order.
25. Reverse the words in a string ("hello world" → "world hello").
26. Find the largest and smallest words in a given string.
27. Check if a given string consists of only digits.
28. Find the shortest word in any sentence.
29. Check if two strings are anagrams (for example, `"listen"` and `"silent"`).
30. Find the longest substring without repeating characters.
31. Implement a custom method to trim leading and trailing spaces (without using `trim()`).
32. Compress a string using counts of repeated characters (*example:* `"aabccccccaaa"` → `"a2b1c5a3"`.
33. Check if a string has all unique characters (kono method use na kore).
34. Implement your own version of `the indexOf()` method.
35. Capitalise the first and last letter of each word in a sentence.
36. Check if two strings are rotations of each other

    For example : `"ABCD"` *and* `"CDAB"` → tahole true asbe.

37. Count and display the frequency of each character in a string.
38. Implement a method to perform basic string compression (remove consecutive duplicates and keep one occurrence only).
39. Remove a specified word from a sentence.
40. Identify and remove all special characters from a string.
41. Split a camelCase string into individual words
    Example: `"thisIsCamelCase"` → `"this Is Camel Case"`.
42. Count the number of digits, letters, and special characters in a string.
43. Convert a Roman numeral string to an integer.
44. Group all vowels at the beginning of a string and consonants at the end
    Preserve relative order.
45. Check if a string is a valid palindrome ignoring spaces and punctuation.
46. Mask all characters of a string except the last four
    Example: `"1234567890"` → `"******7890"`.
47. Remove all duplicate words from a sentence.
48. Count the number of valid email addresses from a sentence.
49. Replace every nth character in a string with a given character.
50. Find the first non-repeating character in a string.
51. Validate whether a string contains a valid IP address format.
52. Implement a function to repeat a string `n` times without using built-in methods.
53. Check if a string ends with a given substring without using `endsWith()`.
54. Count the number of palindromic words in a sentence.
55. Reverse characters in each word of a sentence without changing word order.
56. Identify and count the most frequently occurring word in a paragraph.

Array

1. Create an array of 5 integers and print all elements.
2. Find the length of an array.
3. Access an element at a given index.
4. Change an element at a given index.
5. Loop through an array and print each element.
6. Take 5 integers as input from the user and store them in an array.
7. Print the array in reverse order.
8. Find the sum of all elements in an array.
9. Find the average of all elements in an array.
10. Count how many even and odd numbers are in an array.
11. Check if a given number exists in an array.
12. Find the index of the first occurrence of a given number.
13. Find the largest number in an array.
14. Find the smallest number in an array.
15. Find the second largest number in an array.

16. Sort an array in ascending order (use Arrays.sort()).
17. Sort an array in descending order (built-in methods chara).
18. Replace all negative numbers with zero in an array.
19. Count the number of occurrences of a given number in an array.
20. Remove duplicates from an array (hint: ar ekta array baniye okhane diye dite paro).
21. Merge two arrays into one.
22. Copy elements from one array to another.
23. Swap the first and last elements of an array.
24. Compare two arrays and check if they are equal.
25. Find the common elements between two arrays.
26. Reverse the elements of an array without using a second array. (ektu hard eita for now, pore abar try koiro – two pointers technique ta sikhte paro ekhane)
27. Shift array elements to the left by one position. (left rotation concept chaile sikhte paro)
28. Shift array elements to the right by one position. (right rotation)
29. Find the second smallest number in an array.
30. Count how many numbers are positive, negative, and zero in an array.
31. Find the difference between the largest and smallest elements.
32. Calculate the prefix sum of an array and print it.
33. Check if the array is sorted in ascending order.
34. Check if the array is a palindrome.
35. Find all pairs of elements whose sum is equal to a given number.
36. Find the frequency of each element in an array.
37. Print all unique elements in the array.
38. Insert an element at a specific index in an array (shift elements right).
39. Delete an element from a specific index in an array (shift elements left).
40. Count how many elements are greater than the average of the array.
41. Print all elements which are greater than their immediate neighbours.
42. Move all zeros to the end while maintaining the order of other elements.
43. Find the longest increasing subarray.
44. Reverse the array using the two-pointer approach.
45. Create an array of the cumulative product of elements.
46. Find all elements that appear more than once in the array.
47. Create a new array where each element is the sum of itself and the previous element.
48. Check if all elements in the array are distinct.
49. Count the number of distinct elements in the array
50. Split the array into two halves and swap them.

51. Find the equilibrium index in an array (sum of elements before it equals sum after).
52. Calculate the difference between the sum of even and odd elements.
53. Merge two sorted arrays into a new sorted array

METHODS

1. Create a method to print "Hello, World!" (No parameters, no return value :3)
2. Write a method to calculate the square of a number
3. Develop a method to check if a number is even or odd (RETURN)
4. Implement a method to find the maximum of two numbers
5. Create a method to calculate the factorial of a number (use recursion)
6. Write a method to sum all elements in an integer array
7. Develop a method to find the average of an array of doubles (RETURN)
8. Implement a method to reverse an array (RETURN the new array)
9. Create a method to check if an array contains a specific value
10. Write a method to count the number of vowels in a string
11. Write a method to check if a string is a palindrome
11. Implement a method to convert a string to uppercase
12. Create a method to replace all spaces in a string with hyphens
13. Write a method to count the number of words in a sentence
14. Write a method to find the longest word in a sentence
16. Implement overloaded methods to add two and three integers
17. Create a recursive method to compute the nth Fibonacci number
18. Write a method to calculate the power of a number
19. Implement a method to check if a number is prime
21. Create a method to sort an array of integers in ascending order
22. Write a method to merge two arrays into one
23. Develop a method to remove duplicates from an array
24. Implement a method to find the second largest element in an array
25. Create a method to shift elements of an array to the left by one position
26. Implement a method to count how many times each character appears in a string
27. Count the number of digits in a number using recursion
28. Find the Greatest Common Divisor (GCD) of two numbers using recursion
29. Calculate power using recursion
30. Check if an array is sorted in ascending order using recursion
31. Sum of elements in an array using recursion